

Session 10. Input/Output

1 Instructions

- This exercise **must** be implemented and submitted **as latest 24 hours before** your next laboratory class.
- Rename project as `surname1_surname2_name_lab10`, in lowercase and without accents. Then, export it and upload this file to the corresponding task in Campus Virtual.

2 Outline

Complete functions to read/write from/to files (text and bytes).

- *Load a file* and *Save orders to file* must read from a text file and write to a text file, respectively. In both options, filename must be entered by user.
- *Import from zip* and *Export to zip* must read and write into compressed files (gzip).

3 Reading files

Classes `FileUtil` and `ZipFileUtil` must be implemented to read from files (bytes or text).

```
public List<String> readLines(String pathToTheFile)
    throws FileNotFoundException, IOException {
    try(BufferedReader br = new BufferedReader(new FileReader(pathToTheFile))) {
        // TO DO
    }
    return result;
}
```

```
public List<String> readLines(String pathToTheFile)
    throws FileNotFoundException, IOException {
    try {
        BufferedReader br= new BufferedReader(new FileReader(pathToTheFile));
        // TO DO
    } finally{
        br.close();
    }
    return result;
}
```

Be careful with exceptions thrown in the code and how to handle them:

- `FileNotFoundException` must be handled as a **user error** because the user has provided an incorrect name.

Print a message and give him/her another opportunity to enter the correct filename. Thus, let us allow the exception to propagate back to main loop.

- Other `IOException` must be handled so the program finishes after giving the user an explanation and log the stack trace.

4 Writting files

Classes `FileUtil` and `ZipFileUtil` must also be implemented so they can write into files (text or bytes).

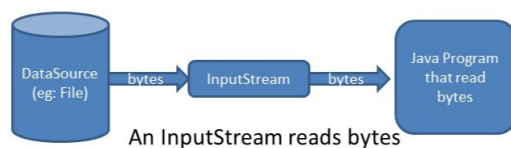
Before writting, it is necessary to transform orders into strings. This serialization (here as a text representation) is done in the `OrderSerializer` class.

5 Chaining streams together to process text and binary files

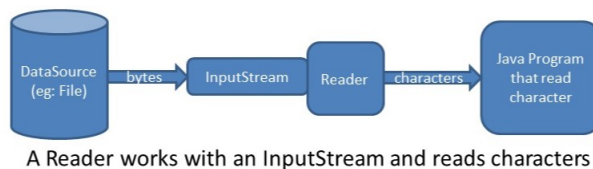
5.1 Text files

Our files are plain text files and we want to read/write characters, not bytes; even more, lines of characters. So, what do you think is the most appropriate solution?

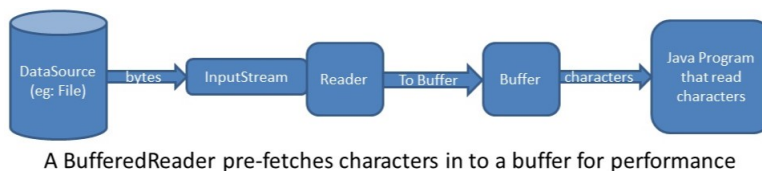
1. `FileInputStream` Reads bytes from File



2. `FileReader` Used to deal with character level.



3. `BufferedReader`: To increase performance. Data to be read will be buffered into memory for quick access.



Unit 7, slides 42, 43.

5.2 Binary files

Lecture 7; slide 51

- Implement it and compare with `FileUtil`.
 - methods in `FileUtil` and `ZipFileUtil` are **essentially the same**. The only difference are the filter chains.

- Think about reuse that code through inheritance.
- In the proposed solution, there is a base class called `BaseFileUtil` with two abstract methods:

```
protected abstract BufferedReader createReaderChain(String inFileName)
    throws FileNotFoundException, IOException;

protected abstract BufferedWriter createWriterChain(String outFileName)
    throws IOException;
```

5.3 Serializer

`saveToFile()` uses a `Serializer` class in order to get a textual representation of a list of objects (`Order`).

6 Create a new `FileLogger` class

Using the actual logger, write a new class `FileLogger` and replace the original one. This class will write the message stream into a text file, with the same format.

Notice that each call to `logger.log(...)` adds a new line to the file.

7 Tests

Write tests for `NewsStand::loadFile()` and `NewsStand::importPublicationsFromZipFile()`. Run them with different files containing any possible valid and invalid lines.